

Unit 4

1 Mark Questions & Answers

File Handling, File modes, File methods

1. **Q:** What is the default mode when opening a file in Python?
A: "r" (read mode).
 2. **Q:** Which file mode is used to add content at the end of a file without overwriting it?
A: "a" (append mode).
 3. **Q:** Which method is used to read one line at a time from a file?
A: `readline()`.
 4. **Q:** What does the `tell()` method do?
A: Returns the current position of the file pointer.
 5. **Q:** Which mode is used to open a file for both reading and writing?
A: "r+".
-

Working on CSV and JSON files:

1. **Q:** What does CSV stand for?
A: CSV stands for *Comma-Separated Values*.
2. **Q:** Which module in Python is used to handle CSV files?
A: The `csv` module.
3. **Q:** Which method is used to write a row into a CSV file?
A: `writerow()` method.
4. **Q:** Which function is used to load data from a JSON file?
A: `json.load()` function.
5. **Q:** Which function converts a Python dictionary into a JSON string?
A: `json.dumps()` function.
6. **Q:** What does `newline=""` do in `open()` while writing CSV files?
A: It prevents extra blank lines from being added between rows in Windows.
7. **Q:** In JSON, what are the data types equivalent to Python's dictionary and list?
A: Dictionary → JSON Object, List → JSON Array.
8. **Q:** Which JSON function is used for pretty printing with indentation?
A: `json.dump(data, file, indent=4)`.

Exceptions, Exception Handling, Raising Exceptions:

1) **Q:** What is an exception in Python?

A: An exception is an error that occurs during program execution and disrupts normal flow.

2) **Q:** Which keywords are used in exception handling in Python?

A: `try`, `except`, `else`, `finally`, `raise`.

3) **Q:** What is the purpose of the `else` block in exception handling?

A: The `else` block runs only if no exception occurs in the `try` block.

4) **Q:** Name two built-in exceptions in Python.

A: `ZeroDivisionError`, `ValueError`.

5) **Q:** What is the use of the `finally` block?

A: It always executes, whether an exception occurs or not (commonly used for cleanup like closing files).

Regular Expressions:

Q1. What is a regular expression in Python?

A1. A regular expression is a sequence of characters that defines a search pattern, often used for string matching and validation.

Q2. Which module in Python supports regular expressions?

A2. The `re` module.

Q3. Which function is used to check if a regex matches the whole string?

A3. `re.fullmatch()`.

Q4. What does the `.` (dot) metacharacter in regex represent?

A4. It matches any single character except newline (`\n`).

Q5. What does `*` mean in regex?

A5. It matches zero or more repetitions of the preceding element.

Q6. Which regex function returns all non-overlapping matches as a list?

A6. `re.findall()`.

Q7. What is the use of `\d` in regex?

A7. It matches any digit (0–9).

2 Mark Questions & Answers

File Handling, File modes, File methods:

- Q:** Differentiate between "w" and "a" file modes.
A: "w" overwrites the file if it exists or creates a new one.
"a" appends data to the end of the file, creating it if it does not exist.
 - Q:** What is the difference between `read()` and `readlines()`?
A: `read()` returns the entire file content as a string.
`readlines()` returns all lines as a list of strings.
 - Q:** What is the purpose of `seek()` method? Give an example.
A: It moves the file pointer to a specific position.
Example: `f.seek(0)` moves the pointer to the beginning of the file.
 - Q:** Write a Python statement to open a file named "data.txt" in binary read mode.
A: `f = open("data.txt", "rb").`
-

Working on CSV and JSON files:

- Q:** Differentiate between `csv.reader()` and `csv.DictReader()`.
A: `csv.reader()` reads each row as a list of strings.
`csv.DictReader()` reads each row as a dictionary, using the first row as keys.
- Q:** Write a Python statement to open a file `data.csv` in append mode.
A: `f = open("data.csv", "a", newline="")`
- Q:** How do `json.dump()` and `json.dumps()` differ?
A: `json.dump()` writes JSON data directly to a file.
`json.dumps()` converts a Python object to a JSON string.
- Q:** Write one line Python code to read JSON data from a file `info.json`.
A: `data = json.load(open("info.json", "r"))`
- Q:** State one advantage of JSON over CSV.
A: JSON supports **nested and hierarchical data**, whereas CSV is limited to tabular data.

Exceptions, Exception Handling, Raising Exceptions

- 1) **Q: Differentiate between syntax error and runtime error.**

A: Syntax error: Errors in code structure, detected before execution (e.g., missing colon).

Runtime error (exception): Errors occurring during execution (e.g., division by zero).

- 2) **Q: Write a Python program using try-except to handle division by zero.**

A: **try:**

```
a = 10 / 0
```

```
except ZeroDivisionError:
```

```
    print("Division by zero is not allowed")
```

- 3) **Q: What is exception propagation?**

A: If an exception is not handled in a function, it propagates back to the caller function until it is handled or program terminates.

- 4) **Q: Give an example of raising an exception manually.**

A: **x = -1**

```
if x < 0:
```

```
    raise ValueError("Negative values not allowed")
```

Regular Expressions:

Q1. Differentiate between `re.search()` and `re.match()`.

A1.

- `re.match()` checks for a match **only at the beginning** of the string.
 - `re.search()` checks for a match **anywhere** in the string.
-

Q2. What does `re.findall()` returns?

A2.

- `re.findall()` returns all matches as a **list of strings**.
-

Q3. Write a regex pattern to validate an Indian mobile number starting with 7, 8, or 9 and having 10 digits.

A3. `pattern = r'\b[789]\d{9}\b'`

Q4. Explain the use of `re.split()` with an example.

A4. `re.split()` splits a string by the occurrences of a pattern.

Example:

```
import re
```

```
print(re.split(' ', 'Python is fun'))
```

```
# Output: ['Python', 'is', 'fun']
```

Q5. How does `re.sub()` work? Give an example.

A5. `re.sub(pattern, repl, string)` replaces all matches of the pattern with `repl`.

Eg 1:

```
import re
```

```
print(re.sub("Python", "Java", "Python programming"))
```

```
Output: Java programming
```

Example 2:

```
import re
```

```
print(re.sub(r'\d', '*', 'Phone: 98765'))
```

```
# Output: Phone: *****
```

5 Mark Questions & Answers

1. **Q:** Explain different file modes in Python with examples.

A:

- "r": Read file (error if not exists).
- "w": Write file (overwrite/create).
- "a": Append data.
- "x": Create new file (error if exists).
- "r+": Read + write existing file.
- "w+": Write + read (overwrite/create).
- "a+": Append + read.

Example:

```
f = open("test.txt", "w")  
f.write("Hello")  
print(f.read("test.txt", "r"))  
f.close()
```

2. Q: Explain any five file methods with examples.

A:

- `read()`: Reads file content.
- `readline()`: Reads one line.
- `write()`: Writes data.
- `seek()`: Moves pointer.
- `tell()`: Returns pointer position.

Example:

```
f = open("test.txt", "w")
f.write("Python\n")
f.close()
```

```
f = open("test.txt", "r")
print(f.readline())
f.close()
```

```
f = open("test.txt", "r")
f.seek(5)      # Moves the file pointer to 5th position
print(f.read()) # Reads the content from 5th position onwards
f.seek(7)
print(f.tell()) # Tells the current position of the file
```

3. Q: Write a Python code to merge two given file contents into a third file.

```
1 file1 = open("sample1.txt", 'w')
2 file1.write("This is python programming lab")
3
4 file2 = open('sample2.txt', 'w')
5 file2.write("\nThis content is not copyrighted")
6
7 file1 = open("sample1.txt", 'r')
8 file2 = open("sample2.txt", 'r')
9 file3 = open("merged.txt", 'w')
10
11 file3.write(file1.read())
12 file3.write(file2.read())
13
14 file3 = open("merged.txt", 'r')
15 print(file3.read())
```

```
This is python programming lab
This content is not copyrighted
```

4. Q: Write a program that reads a file and displays the number of words, number of vowels, blank spaces, lower case letters and uppercase letters.

```
1 file = open('sample1.txt','r')
2 num_words = 0
3 num_vowels = 0
4 num_spaces = 0
5 num_lower = 0
6 num_upper = 0
7
8 for line in file:
9     num_words += len(line.split())
10    for char in line:
11        if char.lower() in 'aeiou':
12            num_vowels = num_vowels + 1
13        if char == ' ':
14            num_spaces = num_spaces + 1
15        if char.islower():
16            num_lower = num_lower + 1
17        if char.isupper():
18            num_upper = num_upper + 1
19
20 print(f"Number of words: {num_words}")
21 print(f"Number of vowels: {num_vowels}")
22 print(f"Number of spaces: {num_spaces}")
23 print(f"Number of lowercase letters: {num_lower}")
24 print(f"Number of uppercase letters: {num_upper}")
```

```
Number of words: 5
Number of vowels: 7
Number of spaces: 4
Number of lowercase letters: 25
Number of uppercase letters: 1
```

Exceptions, Exception Handling, Raising Exceptions:

5. Q: Explain the flow of execution in try, except, else, and finally blocks with an example.

A:

- try: Contains code that may raise an exception.
- except: Handles the exception.
- else: Executes only if no exception occurs.
- finally: Always executes.

Example:

```
a = int(input("enter the first no: "))
b = int(input("enter the second no: "))
try:
    x = a / b # This will raise a ZeroDivisionError
except:
    print("can't divide by zero")
else:
    print(f"The result is: {x}")
finally:
    print("Execution finished.")
```

Explanation: If num=0, exception occurs → except runs → finally runs.
If num=5, no error → else runs → finally runs.

6. Q: List and explain any four built-in exceptions in Python with examples.

A: i) ZeroDivisionError: Raised when dividing by zero.
Eg: print(5/0)
ii) **ValueError:** Raised when invalid value is passed.
Eg: int("abc")
iii) **IndexError:** Raised when accessing an invalid list index.
Eg: lst = [1,2,3]
print(lst[5])
iv) **TypeError:** Raised when operation is applied to an invalid type.
Eg: print("5" + 2)

7 Q: Explain except clause with multiple exceptions.

```
try:
    x = int(input("Enter a number: "))
    y = 10 / x
    print("Result:", y)
except (ValueError, ZeroDivisionError):
    print("Error occurred! Either invalid input or division by zero.")
```

8 Q: Explain how to create user-defined exceptions.


```

1 # Example of raising an exception
2 class InvalidMarksError(Exception):
3     pass
4
5 try:
6     marks = int(input('enter the marks '))
7     if marks > 100 or marks < 0:
8         raise InvalidMarksError("Marks should be between 0 and 100")
9     else:
10        print("Marks are valid")
11 except InvalidMarksError as e:
12     print("Error:", e)
13 except ValueError:
14     print("Only Numbers are allowed")

```

enter the marks 200
 Error: Marks should be between 0 and 100

9Q: What happens if except clause is written without any Exception type? Explain with an example.

Answer: If you write an **except** clause without specifying any exception type, it will catch all exceptions (no matter what error occurs).

Eg:

```

try:
    x = int("abc")    # This will raise ValueError
    y = 10 / 0        # This would raise ZeroDivisionError
except ValueError:
    print("Value error occurred!")
except:               # Catches all exceptions other than ValueError
    print("error occurred!")

```

10Q: Write about Errors and Exception handling in python. or

11 Q: Define error and exception. Differentiate them.

A. Error

- An **error** indicates a problem in the program that makes it impossible to run.
- Errors are mostly **syntax-related** or **logical mistakes** made by the programmer.
- Errors stop the execution of the program **before it runs properly**.

♦ Exception

- An **exception** is an **event that occurs during program execution** and disrupts the normal flow of instructions.

- Exceptions usually happen due to **runtime issues** (like division by zero, accessing an undefined variable, file not found, etc.).
- Exceptions can be **handled** using `try-except`.

Differences between Error and Exception

Aspect	Error	Exception
Definition	Mistakes in program code (syntax/logic)	Events that occur during program execution
When it occurs	During compilation/parsing (before execution)	During execution (runtime)
Examples	<code>SyntaxError</code> , <code>IndentationError</code>	<code>ZeroDivisionError</code> , <code>ValueError</code> , <code>FileNotFoundError</code>
Can it be handled?	No , must be fixed in code	Yes , can be handled with <code>try-except</code>
Cause	Programmer mistakes (typos, wrong syntax)	Invalid operations or unexpected runtime conditions
Recovery	Program cannot continue unless fixed	Program can continue if handled properly

Regular Expressions:

12 Q: Describe the core functions and methods of `re` module

A. Core functions of `re` module:

- >`match`: Returns a `Match` object if there is a match else returns `None`.
- >`search`: Returns a `Match` object if there is a match anywhere in the string.
- >`findall`: Returns a list containing all matches.
- >`split`: Returns a list where the string has been split at each match.
- >`sub`: Replaces one or many matches with a string

Methods: `group`, `start`, `end`, `span`

- `group()` → Returns the matched string.
- `start()` → Start index of match.
- `end()` → End index of match.
- `span()` → Tuple of (`start`, `end`).

13 Q: Does regular expression pattern enable matching of multiple strings. Justify.

A. Yes, **regular expression (regex) patterns enable matching of multiple strings** because they are not fixed string literals – they are **patterns** that describe a set of strings that share a common structure.

Justification

1. A **regex pattern** defines **rules** (using special characters like `.`, `*`, `+`, `|` etc.) rather than exact text.
2. These rules allow a single regex pattern to match **different variations of strings** at once.
3. This makes regex powerful for **search, validation, and text extraction**.